

AGRIVISTA FIELD

Dossier de conception

Mise en situation professionnelle - Développement full stack mobile Flutter

Formation	Master BIHAR
Technologie	Flutter - Dart 3
Projet	Application mobile de gestion d'interventions terrain
Auteur	ARGAILLOT Dorian
Session	2025-2026
Version de conception	1.1 - extensions optionnelles retenues

Finalité du document

Présenter et justifier la conception d'AgriVista Field : socle obligatoire validé puis intégration de quatre extensions optionnelles destinées à renforcer la valeur métier et la démonstration technique : tableau de bord, actualisation/tri, compte rendu terrain photo/note, thème clair-sombre et adaptation tablette.

1. Reformulation du besoin et périmètre retenu

1.1 Contexte professionnel

AgriVista installe et maintient des stations de capteurs connectés pour des exploitations agricoles et viticoles corses. Les techniciens interviennent sur le terrain, parfois dans des zones où la couverture réseau est faible ou intermittente. L'application interne « AgriVista Field » remplace une gestion papier des interventions et fournit un outil mobile fiable sur Android et iOS.

1.2 Besoin métier reformulé

Le technicien doit consulter les interventions de la source JSON de référence, les rechercher et les filtrer, ouvrir leur détail et faire évoluer leur statut. Les changements sont conservés localement. Le socle obligatoire étant validé, la version 1.1 étend l'application avec des fonctions orientées pilotage, confort terrain et adaptation d'interface, sans ajouter de backend ni modifier le JSON distant.

1.3 Objectifs fonctionnels

- Maintenir l'intégralité du socle obligatoire déjà validé.
- Ajouter un tableau de bord de synthèse par statut et priorité.
- Permettre un pull-to-refresh et le tri des interventions.
- Associer localement une note et une photo à une intervention.
- Proposer un thème système / clair / sombre mémorisé localement.
- Adapter la navigation et la mise en page aux tablettes.

1.4 Périmètre retenu

Élément	Décision	Précision
Socle obligatoire	Validé	JSON, liste, filtres, détail, statuts, Hive, états UI, profil.
Tableau de bord	Inclus - extension	Compteurs par statut et priorité, accès aux listes filtrées.
Pull-to-refresh / tri	Inclus - extension	Actualisation distante + tri date, priorité et statut.
Photo + note	Inclus - extension	Compte rendu terrain local attaché à une intervention.
Thème clair/sombre	Inclus - extension	Mode système, clair ou sombre, persistant.
Responsive tablette	Inclus - extension	Navigation et layout adaptés aux grands écrans.
Backend / synchro	Exclu	Toujours hors périmètre de cette version.
Login / carte / i18n	Exclu	Non retenus parmi les options disponibles.

Hypothèse de conception

Les extensions réutilisent les données et les frontières existantes. Le JSON reste la source de référence en lecture seule. Les données ajoutées sur le terrain sont exclusivement locales et ne sont jamais injectées dans l'historique distant.

2. Acteurs, cas d'usage et user stories

2.1 Acteur principal

Le technicien de terrain reste l'unique acteur humain. Le JSON distant constitue la source externe de référence ; le stockage local conserve statuts, compte rendu et préférences d'affichage.

2.2 User stories principales et extensions

ID	Acteur	User story
US01-09	Technicien	Socle validé : charger, rechercher, filtrer, consulter, progresser, restaurer, réessayer, consulter le profil.
US10	Technicien	Visualiser une synthèse de mes interventions afin d'identifier rapidement la charge de travail.
US11	Technicien	Actualiser la liste afin de récupérer les dernières données disponibles.
US12	Technicien	Trier les interventions afin de les organiser par date, priorité ou statut.
US13	Technicien	Ajouter une note terrain afin de conserver une information liée à l'intervention.
US14	Technicien	Associer une photo locale afin de documenter visuellement l'intervention.
US15	Technicien	Choisir un thème système, clair ou sombre selon mes conditions d'utilisation.
US16	Technicien	Utiliser une interface adaptée à une tablette ou à un écran large.

2.3 Cas d'usage complémentaires

ID	Cas d'usage	Résultat attendu
UC07	Consulter le tableau de bord	Compteurs cohérents et navigation vers la liste filtrée.
UC08	Actualiser les interventions	Nouveau JSON chargé puis refusionné avec les données locales.
UC09	Trier les interventions	Ordre recalculé sans modifier la source.
UC10	Ajouter/modifier une note	Texte local persistant par intervention.
UC11	Ajouter/remplacer une photo	Fichier local persistant et chemin conservé.
UC12	Changer le thème	Apparence mémorisée après redémarrage.
UC13	Utiliser le mode tablette	Layout adapté sans perte de fonctionnalité.

3. Scénarios principaux et parcours utilisateur

3.1 Parcours enrichi

1. L'application charge le JSON distant, relit les données locales et expose l'état fusionné via Riverpod.
2. Le tableau de bord affiche les compteurs ; une carte de statut peut ouvrir directement la liste avec le filtre correspondant.
3. La liste peut être recherchée, filtrée, triée puis actualisée par pull-to-refresh.
4. Depuis le détail, le technicien fait évoluer le statut, saisit une note et peut associer une photo locale.
5. Le profil permet de choisir l'apparence système, claire ou sombre.
6. Sur tablette, l'application adapte la navigation et privilégie une présentation master/detail.

3.2 Scénario - actualisation distante

1. Le technicien déclenche le pull-to-refresh depuis la liste.
2. Le RemoteDataSource effectue un nouveau GET sur le JSON officiel.
3. Le repository relit les surcharges et données locales avant de reconstruire la liste.
4. Les statuts locaux restent prioritaires ; les notes et photos restent associées à leur identifiant.
5. L'UI repasse de loading à data, ou affiche une erreur exploitable si l'actualisation échoue.

3.3 Scénario - compte rendu terrain

1. Le technicien ouvre une intervention et saisit ou modifie sa note.
2. Lorsqu'une photo est choisie, elle est copiée dans le stockage permanent de l'application.
3. Hive enregistre la note et le chemin du fichier, pas le contenu binaire de l'image.
4. Après fermeture puis relance, le compte rendu est restauré localement.

3.4 Scénarios d'erreur complémentaires

Situation	Traitement	Comportement UI
Échec pull-to-refresh	Failure existante	Conserver les données déjà valides et informer l'utilisateur.
Photo inaccessible	Erreur locale ciblée	Ne pas supprimer le reste du compte rendu ; message explicite.
Écriture note/photo impossible	LocalStorageFailure	Ancienne valeur conservée, aucune fausse réussite.
Préférence thème illisible	Erreur de préférence	Revenir au mode système de façon contrôlée.

Parcours synthétique

Dashboard → liste / recherche / filtres / tri / refresh → détail → statut + compte rendu terrain → profil / apparence. Sur tablette, la liste et le détail peuvent être affichés simultanément.

4. Maquettes / wireframes des écrans clés

Les wireframes ci-dessous complètent ceux du socle. Ils définissent la hiérarchie fonctionnelle ; le design final reste piloté par Material 3 et le ColorScheme de l'application.

Extensions UX retenues

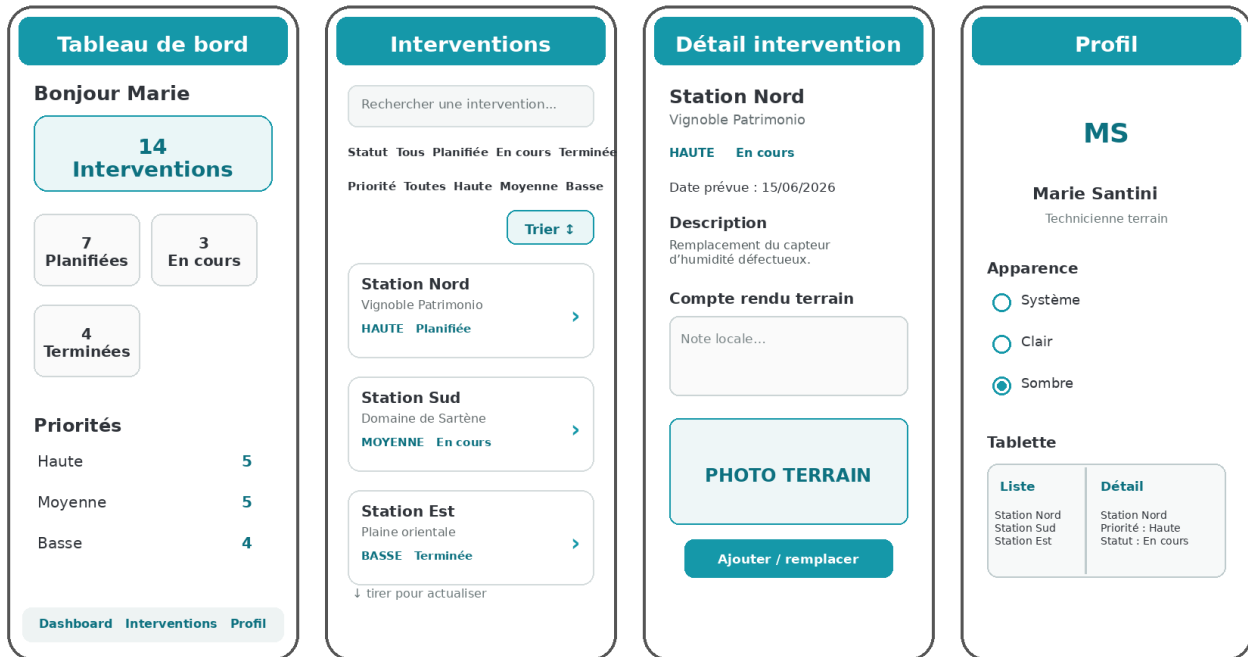


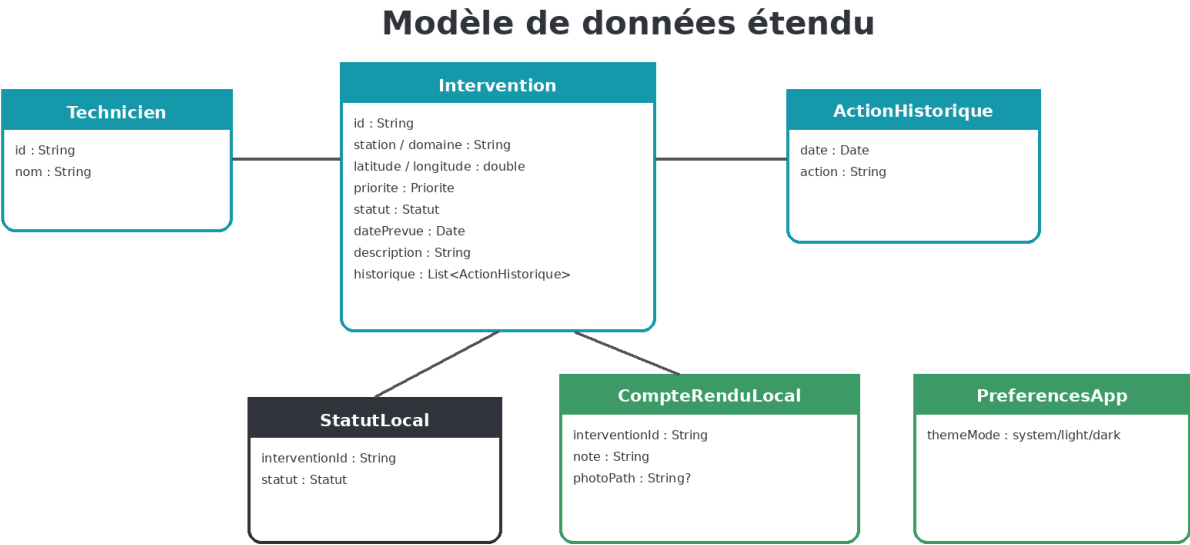
Figure 2 - Dashboard, liste enrichie, compte rendu terrain et préférences / tablette.

Principes UX retenus

- La NavigationBar mobile évolue vers Dashboard / Interventions / Profil.
- Le dashboard donne une synthèse actionnable, sans dupliquer la donnée métier.
- Le pull-to-refresh reste un geste naturel de la liste ; le tri est explicite et réversible.
- La note et la photo sont regroupées sous un bloc « Compte rendu terrain » dans le détail.
- Le choix de thème est placé dans le profil.
- Le layout tablette privilégie une lecture master/detail et peut employer NavigationRail.

5. Modèle de données

Le Domain conserve les concepts du JSON et ajoute un objet de compte rendu local, sans dépendre de Hive ni du système de fichiers. Les préférences d'apparence sont des données applicatives locales.



Le fichier image est stocké sur le système de fichiers ; Hive conserve uniquement son chemin.

Figure 3 - Entités métier et données locales ajoutées par les extensions.

5.1 Entités et données locales

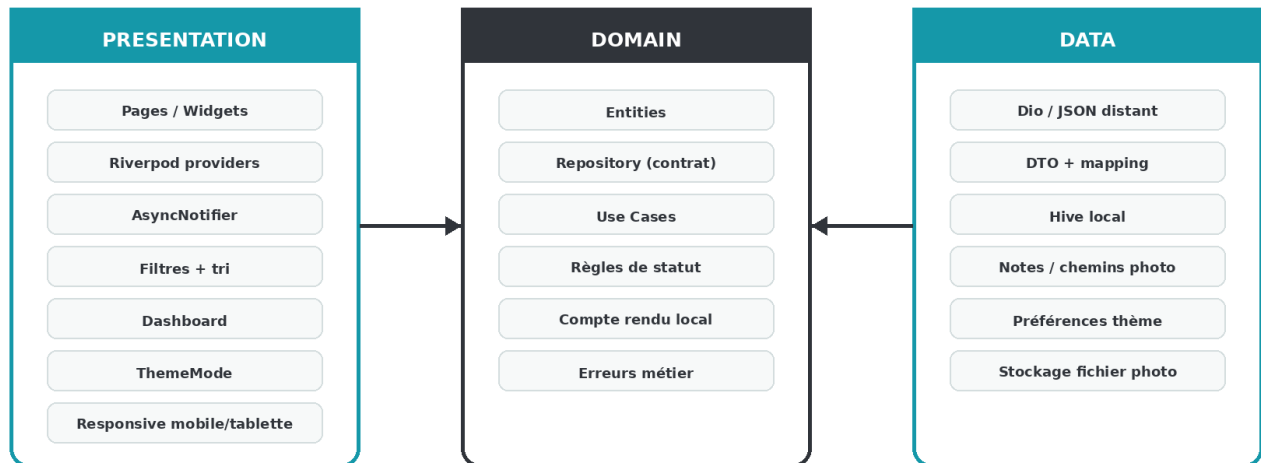
Élément	Attributs principaux	Responsabilité
Intervention	socle inchangé	Objet métier principal issu du JSON.
StatutLocal	interventionId, statut	Surcharge locale déjà existante.
CompteRenduLocal	interventionId, note, photoPath	Note et référence de photo persistées localement.
PreferencesApp	themeMode	Préférence système / clair / sombre.
DashboardSummary	compteurs dérivés	Vue calculée depuis la liste, non persistée.

5.2 Règles complémentaires

- La note et la photo n'altèrent jamais le JSON ni l'historique distant.
- Le fichier image est stocké dans un emplacement persistant de l'application ; Hive ne conserve que son chemin.
- Une donnée locale est rattachée à l'identifiant de l'intervention ; une entrée orpheline n'invente aucune intervention.
- Les compteurs du dashboard sont recalculés depuis l'état Riverpod après chaque changement de statut.

6. Architecture logicielle et gestion d'état

Architecture cible - socle + extensions



Règle de dépendance : Presentation et Data dépendent du Domain, jamais l'inverse.

Les extensions réutilisent les mêmes frontières : dashboard dérivé de l'état, préférences et données terrain persistées localement.

Figure 4 - Clean Architecture conservée pour les extensions.

6.1 Responsabilités étendues

Couche	Ajouts liés aux extensions	Dépendances
Presentation	Dashboard, état de tri, ThemeMode, responsive mobile/tablette, UI compte rendu.	Flutter + Riverpod
Domain	Compte rendu local et use cases associés ; règles existantes inchangées.	Dart pur
Data	Hive note/photo/préférences, gestion de fichier photo, repository local.	Détails techniques

6.2 Arborescence cible

```

lib/
├── app/
├── core/
├── features/
│   ├── dashboard/
│   ├── interventions/
│   │   ├── domain/
│   │   ├── data/
│   │   └── presentation/
│   └── profile/
└── main.dart
    
```

6.3 Choix Riverpod

Le provider principal reste la source de vérité des interventions. Le dashboard dérive ses compteurs de cet état. Le Notifier de filtres accueille le tri. Un provider dédié gère ThemeMode et sa persistance. Les opérations note/photo passent par des use cases et repositories, sans accès Hive ou fichiers depuis les widgets.

7. Choix techniques argumentés

Sujet	Choix	Justification	Alternative écartée
Dashboard	État dérivé Riverpod	Aucune persistance ; compteurs toujours synchronisés avec la liste.	Stockage de compteurs : inutile et source d'incohérences.
Refresh	RefreshIndicator + recharger()	Réutilise le flux existant et refusionne JSON/Hive.	Retry automatique : hors choix UX retenu.
Tri	Notifier Presentation	Préoccupation d'affichage locale, combinable aux filtres.	Tri serveur : aucun backend.
Note	Hive String	Besoin texte simple, local et persistant.	Base SQL : surdimensionnée.
Photo	image_picker + path_provider	Sélection/capture puis copie dans le stockage permanent.	Binaire dans Hive : volume et maintenance inutiles.
Thème	ThemeMode + Hive	Choix explicite et persistant, conforme Material 3.	Réglage non persistant : expérience incohérente.
Tablette	LayoutBuilder	Adaptation selon largeur sans multiplier les variantes.	UI fixe étirée : peu adaptée aux grands écrans.

7.1 Packages majeurs prévus

Package	Rôle
flutter_riverpod	État, DI, dashboard dérivé, tri et préférences.
dio	Lecture HTTP et pull-to-refresh.
freezed / json_serializable	DTO et désérialisation typée.
hive_flutter	Statuts, notes, chemins photo et préférences.
image_picker	Sélection / capture d'une photo locale.
path_provider	Répertoire persistant de l'application pour les fichiers photo.

7.2 Gestion d'erreurs

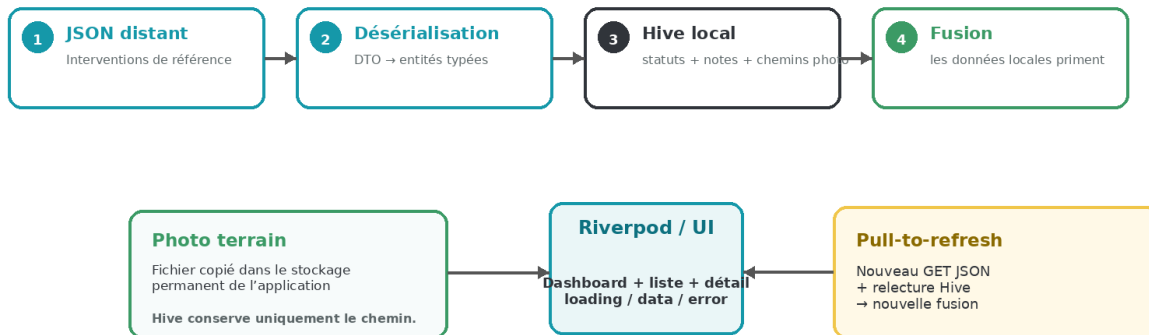
La stratégie AppFailure reste inchangée. Les nouveaux accès locaux doivent convertir les erreurs de fichier ou de préférence en échecs explicites. Une erreur liée à une photo ou à une note reste locale à l'action lorsque les données principales sont encore valides.

Principe

Une extension ne doit jamais dégrader le socle obligatoire : les 86 tests existants restent une base de non-régression et de nouveaux tests seront ajoutés pour chaque fonctionnalité.

8. Stratégie de persistance locale étendue

Flux de données et persistance locale étendus



Le JSON reste inchangé. Le cache complet reste hors périmètre : un chargement réseau est toujours nécessaire pour reconstruire la liste.

Préférences de thème : `box app_preferences` • `theme_mode` → `system` / `light` / `dark`

Figure 5 - Fusion distante, données terrain locales, préférences et actualisation.

8.1 Structures locales

Stockage	Clé	Valeur
intervention_statuses	interventionId	planifiée / en_cours / terminée
intervention_notes	interventionId	note terrain
intervention_photos	interventionId	chemin du fichier local
app_preferences	theme_mode	system / light / dark

8.2 Écriture d'un compte rendu

1. La note est enregistrée directement dans le stockage local.
2. La photo sélectionnée est copiée dans le stockage permanent de l'application.
3. Le chemin du fichier est persisté après réussite de la copie.
4. En cas d'échec, l'ancienne donnée valide reste affichée.

8.3 Actualisation et limites

- Le pull-to-refresh recharge le JSON puis réapplique toutes les données locales connues.
- Les statuts, notes, photos et préférences survivent au redémarrage.
- Le JSON complet n'est toujours pas mis en cache : un réseau est requis pour reconstruire la liste.
- La synchronisation serveur des données locales reste hors périmètre.

9. Traçabilité des exigences et critères de validation

Exigence	Composants prévus	Critère de validation
Socle obligatoire	Architecture et code existants	Tous les tests historiques continuent de passer.
Dashboard	DashboardPage + provider dérivé	Compteurs cohérents et mis à jour après transition.
Pull-to-refresh	RefreshIndicator + recharger()	Nouveau GET, surcharges locales toujours appliquées.
Tri	État de tri Presentation	Date / priorité / statut ordonnent correctement la liste.
Note terrain	Use case + Hive	Note restaurée après fermeture / relance.
Photo locale	image_picker + stockage fichier + Hive	Photo toujours accessible après relance.
Thème	ThemeMode provider + app_preferences	Système / clair / sombre conservé après redémarrage.
Responsive tablette	LayoutBuilder + layout large	Aucun overflow, navigation et détail adaptés.
Versionnement	Git + tag v1.0-socle	Un commit significatif par extension.

10. Conclusion de conception

Le socle obligatoire d'AgriVista Field est stabilisé et tagué avant extension. La version 1.1 conserve la Clean Architecture et la stratégie JSON en lecture seule, tout en ajoutant quatre axes différenciants : pilotage par tableau de bord, actualisation et tri, compte rendu terrain local photo/note, et adaptation de l'interface au contexte d'utilisation via thème et tablette. Ces fonctionnalités restent indépendantes d'un backend et doivent être intégrées par étapes afin de préserver la non-régression du socle.

Stratégie de réalisation

Chaque extension fera l'objet d'une étape Codex dédiée, de tests automatisés et d'une validation manuelle avant commit. Le dossier technique ne sera mis à jour qu'après implémentation effective afin de décrire le code réellement livré.

Annexe - URL de la source de données

<https://utrera.ludovic.aflokkat-projet.fr/getInterventions.json>